Field-Correlation Cortex — real-time OSINT × Field corroboration

SPEC-TH-041 v0.1 DRAFT Jeremie Nunez

1 Context

The OSINT nano-swarm emits hypothesis edges at scale from open sources — noisy, high-volume, low-trust. A separate high-trust field stream carries events from a tactical data-link, a sensor-fusion bus, friendly-force tracking, mission debriefs, and C2 feeds — authoritative, low-volume, restricted. The value of the system is in fusing the two: OSINT raises hypotheses, field events confirm or refute them, and the knowledge graph reflects the fusion in quasi-real-time. This spec defines the **field-correlation cortex**, a new cortex subscribed to the tactical message bus (abstract over Kafka, ZeroMQ, MQTT) that consumes incoming field events, finds matching edges, and invokes `ConfidenceService` (SPEC-TH-040) to promote or demote them under a strict latency budget.

2 Goals

- A new cortex `field-correlation` implementing the `Cortex` interface, registered alongside the other cortices.
- Subscribes to a tactical message bus behind a transport-agnostic `FieldStreamSubscriber` interface (Kafka, ZeroMQ, MQTT adapters).
- Matches incoming field events to candidate edges via `(subject, relation, object)` keying with configurable fuzziness (entity aliases, temporal windows).
- Invokes `ConfidenceService.promote` or `demote` on matches. No direct writes to the edge table — always through the service (SPEC-TH-040 I-5).
- Meets a per-event latency budget: $<$ **500 ms end-to-end** for *critical* events, from bus arrival to provenance event emission.
- Emits per-event metrics (`match_hit`, `match_miss`, `contradiction`, `latency_ms`) for observability.

3 Non-Goals

Non-Goal

This spec does *not* define the wire-level payload of the tactical data-link, the authentication / encryption of the bus, or the classification-level controls on field ingestion — those live in deployment-side documents. It also does *not* re-specify the confidence model; that is SPEC-TH-040. Multi-hop inference (e.g. promoting edges two hops away from a field-matched edge) is out of scope for v0.1.

4 Invariants

Invariant

I-1 — Service-only writes. The cortex never mutates edge rows directly. Every promotion or demotion goes through `ConfidenceService`, which enforces the per-class invariants from SPEC-TH-040.

Invariant

I-2 — Field dominance. A field event always overrides OSINT confidence on a matching edge. The cortex must *not* skip a promotion or demotion because of an older OSINT signal; field wins.

Invariant

I-3 — OSINT cannot demote FIELD_HIGH. The field-correlation cortex rejects and logs any request — internal or upstream — that would demote `FIELD_HIGH` from a non-field actor.

Invariant

I-4 — Latency budget enforcement. Every event carries a deadline derived from its `priority` (`critical`, `routine`, `background`). A processing path that exceeds its deadline emits a `LatencyBreach` event and continues; it does *not* silently drop the event, and it does *not* block the bus subscriber.

Invariant

I-5 — At-least-once correlation. The cortex processes each field event at least once. It is idempotent under replay: re-applying the same event to the same edge is a no-op in terms of class and confidence value.

Invariant

I-6 — No cross-classification leakage. Field events remain on their stream partition. A high-classification field event never produces a log line, metric label, or provenance field value that leaks content to a lower-classification observability sink. (Implementation: stream-tagged sinks; enforced by policy, asserted in tests.)

5 Latency SLO table

Per-event deadline from bus arrival to `EdgeProvenanceEvent` emission (p99 targets).

Priority	Examples	p50	p99 deadline
`critical`	Engagement report, IFF anomaly, imminent-threat friendly-force track	150 ms	**500 ms**
`routine`	Track update, EW contact, routine C2 status change	500 ms	**2,000 ms**
`background`	Post-mission debrief ingest, batched sensor-fusion replay	3 s	**10 s**

Budget is split approximately as: bus deserialize $\leq 10\%$, candidate-edge lookup $\leq 40\%$, confidence service call $\leq 30\%$, provenance emission $\leq 20\%$.

6 Interface

```ts
// packages/thalamus/src/cortices/field-correlation/types.ts

import type { Cortex, CortexInput, CortexOutput } from "../types";
import type { ConfidenceService, SourceClass } from "../confidence";

export type FieldStream =
  | "tactical-data-link"
  | "sensor-fusion-bus"
  | "friendly-force-tracking"
  | "c2-feed"
  | "mission-debrief"
  | "ew-contact"
  | "tactical-sigint";

export type FieldEventPriority = "critical" | "routine" | "background";

export interface FieldEvent {
  id: string;
  stream: FieldStream;
  priority: FieldEventPriority;
  receivedAt: Date;
  subject: { type: string; id: string; aliases?: string[] };
  relation: string;
  object:  { type: string; id: string; aliases?: string[] };
  outcome: "confirms" | "contradicts" | "partial";
  // Opaque payload retained for audit; never logged at lower
     classification.
  payload: Record<string, unknown>;
}

export interface FieldStreamSubscriber {
  subscribe(
    streams: FieldStream[],
    onEvent: (event: FieldEvent) => Promise<void>,
  ): Promise<void>;
  close(): Promise<void>;
}

export interface CorrelationResult {
  eventId: string;
  matchedEdgeIds: number[];
  outcome:
    | { kind: "promoted"; to: SourceClass }
    | { kind: "demoted";  to: SourceClass }
    | { kind: "no-match" }
    | { kind: "latency-breach"; budgetMs: number; actualMs: number };
  latencyMs: number;
}

export interface FieldCorrelationCortex extends Cortex {
  name: "field-correlation";
  // Streaming entry point; orchestrator starts this at boot.
  start(subscriber: FieldStreamSubscriber): Promise<void>;
  stop(): Promise<void>;
  // Cortex.execute is reused for replay / back-fill batches.
  execute(input: CortexInput): Promise<CortexOutput>;
}
```

7 Registration

The cortex is registered in `CortexRegistry` by a `field-correlation.md` skill file declaring its name, description, and the SQL helper used for candidate-edge lookup. At boot, the orchestrator resolves `FieldStreamSubscriber` from DI (bus transport swappable) and calls `cortex.start(subscriber)`.

8 Scenarios

Scenario: Critical field event confirms an OSINT hypothesis

Given an `OSINT_CORROBORATED` edge at `confidence = 0.55`
When a `critical` friendly-force tracking event arrives on the bus confirming the triple
Then the cortex calls `ConfidenceService.promote` with `field-match` evidence
And the edge becomes `FIELD_HIGH`
And end-to-end latency is below 500 ms (p99)
And a `correlation.match_hit` metric is emitted with `priority=critical`

Scenario: Routine field event with partial match

Given an `OSINT_UNCORROBORATED` edge at `confidence = 0.30`
When a `routine` sensor-fusion event arrives with outcome `partial`
Then the edge is promoted to `FIELD_LOW`
And `confidence` is set to `0.75`
And end-to-end latency is below 2,000 ms

Scenario: Field event contradicts a FIELD_LOW edge

Given a `FIELD_LOW` edge backed by an older field event
When a contradicting field event arrives on the same stream
Then the cortex calls `ConfidenceService.demote` with `field-contradiction` evidence
And the edge drops to `OSINT_UNCORROBORATED` with `confidence` ≤ 0.20
And the edge is flagged for Sweep analyst review

Scenario: No matching edge

Given an incoming field event whose triple does not match any existing edge
When the cortex runs the candidate-edge lookup
Then no mutation is performed
And a `correlation.match_miss` metric is emitted
And the event is retained on an unmatched-events queue for later correlation (bounded, TTL-gated)

Scenario: Latency breach on critical event

Given a `critical` event with budget 500 ms
When the candidate-edge lookup exceeds the budget due to DB contention
Then the cortex still completes the promotion
And emits a `LatencyBreach` event with `budgetMs=500` and actual duration
And the observability bus records an SLO violation
And the event is not dropped

Scenario: OSINT attempts to demote FIELD_HIGH

Given a `FIELD_HIGH` edge
When a late-arriving OSINT corroboration contradicts the field-confirmed relation
Then the cortex refuses the demotion
And logs a `rejected-demotion` event with actor `osint-swarm`
And the edge class and confidence remain unchanged

9 Acceptance Criteria

Acceptance Criterion AC-1

The cortex registers under name `field-correlation` and is discoverable via `CortexRegistry.names()`.

Acceptance Criterion AC-2

Starting the cortex with a mock `FieldStreamSubscriber` produces a delivered `critical` event routed through `ConfidenceService.promote`, ending in a `FIELD_HIGH` edge within 500 ms (p99 over 1,000 synthetic events).

Acceptance Criterion AC-3

A `routine` event completes end-to-end within 2,000 ms (p99 over 1,000 synthetic events); `background` within 10 s.

Acceptance Criterion AC-4

A contradicting field event demotes a matching `FIELD_*` edge to `OSINT_UNCORROBORATED` with `confidence` ≤ 0.20.

Acceptance Criterion AC-5

A request that would demote `FIELD_HIGH` from a non-field actor is rejected; edge state is unchanged; a `rejected-demotion` metric is emitted.

Acceptance Criterion AC-6

Replaying the same `FieldEvent.id` produces no additional class or confidence change (idempotence under at-least-once delivery).

Acceptance Criterion AC-7

An event with no matching edge emits `correlation.match_miss` and persists to the bounded unmatched-events queue with a TTL.

Acceptance Criterion AC-8

Every processed event emits an `EdgeProvenanceEvent` via `ConfidenceService` (when it mutates) with `actor = field-correlation`.

Acceptance Criterion AC-9

A latency breach emits a `LatencyBreach` event tagged with `priority` and does not cause event loss.

Acceptance Criterion AC-10

Observability sinks at lower classification never receive `FieldEvent.payload` content (asserted by a leakage test that inspects all emitted log lines and metric labels against an allow-list).

10 Traceability

AC	Test file (planned)	Test name
AC-1	`packages/thalamus/tests/field-correlation/registry.spec.ts`	`field-correlation`
AC-2	`packages/thalamus/tests/field-correlation/latency-critical.spec.ts`	`p99 < 500ms on cr`
AC-3	`packages/thalamus/tests/field-correlation/latency-routine.spec.ts`	`p99 < 2s routine`
AC-4	`packages/thalamus/tests/field-correlation/demote.spec.ts`	`field contradicti`
AC-5	`packages/thalamus/tests/field-correlation/invariants.spec.ts`	`OSINT cannot demo`
AC-6	`packages/thalamus/tests/field-correlation/idempotence.spec.ts`	`replay is no-op`
AC-7	`packages/thalamus/tests/field-correlation/no-match.spec.ts`	`miss routes to un`
AC-8	`packages/thalamus/tests/field-correlation/provenance.spec.ts`	`provenance actor`
AC-9	`packages/thalamus/tests/field-correlation/latency-breach.spec.ts`	`breach emits even`
AC-10	`packages/thalamus/tests/field-correlation/leakage.spec.ts`	`payload not in lo`

11 Open Questions

- Candidate-edge lookup: pgvector similarity on aliases vs. exact-key hash index? Likely hybrid — exact first, vector fallback.
- Unmatched-events queue: Redis stream vs. a dedicated table with a TTL sweep? Leaning Redis stream with a length cap.
- Per-stream priority override: can a `mission-debrief` event ever be `critical`? Current draft: priority is event-level, stream is metadata.
- Back-pressure policy when bus rate exceeds worker capacity: shed `background` first, then `routine`; never `critical`.
- Classification policy for the unmatched-events queue: per-stream partition keys; test under I-6.